

Lab6 进程调度

一、练习1：理解调度器框架的实现

问题分析：

调度类结构体 sched_class 的分析：

sched_class结构体的定义：

代码块

```
1  struct sched_class
2  {
3      // the name of sched_class
4      const char *name;
5      // Init the run queue
6      void (*init)(struct run_queue *rq);
7      // put the proc into runqueue, and this function must be called with
      rq_lock
8      void (*enqueue)(struct run_queue *rq, struct proc_struct *proc);
9      // get the proc out runqueue, and this function must be called with rq_lock
10     void (*dequeue)(struct run_queue *rq, struct proc_struct *proc);
11     // choose the next runnable task
12     struct proc_struct *(*pick_next)(struct run_queue *rq);
13     // dealer of the time-tick
14     void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc);
15     /* for SMP support in the future
16     *   load_balance
17     *       void (*load_balance)(struct rq* rq);
18     *   get some proc from this rq, used in load_balance,
19     *   return value is the num of gotten proc
20     *   int (*get_proc)(struct rq* rq, struct proc* procs_moved[]);
21     */
22 };
```

函数指针的作用和调用时机

- init:
 - 作用：初始化一个run_queue的内部结构，为该run_queue设置初始参数。
 - 调用时机：在创建或重置run_queue被调用一次。

- enqueue:
 - 作用：把一个proc_struct放入run_queue，并更新与调度相关的元数据。
 - 调用时机：当某个进程从阻塞/睡眠/就绪状态变为可运行需要被放入某个run_queue时调用。
- dequeue:
 - 作用：从run_queue中移除指定的proc_struct，并更新元数据。
 - 调用时机：当进程离开就绪队列、被do_wait/do_kill回收、或迁移到其他队列时调用。
- 结构体：proc_next:
 - 作用：从run_queue中选择并返回下一个应被调度的proc_struct
 - 调用时机：调度点被触发时用来决定下一要运行的进程；在抢占或时间片耗尽后被调用。
- proc_tick:
 - 作用：处理每个时间片定时器到来时对run_queue中某个运行进程的维护工作。
 - 调用时机：在时钟中断处理或调度周期内对当前运行的进程进行一次“滴答”处理；通常由定时器中断上下文或调度器在每个tick后调用。

为什么需要将这些函数定义为函数指针，而不是直接实现函数：

- 可扩展性与模块化：函数指针形成“调度类接口”（像面向对象的虚函数），允许在同一内核中实现多个不同的调度策略（RR、FIFO、stride、CFS 风格等），并在运行时或编译时选择或注册不同的 `sched_class` 实现而无需修改调度核心。
- 多态行为：不同 `run_queue`（例如不同 CPU、本地/全局队列或实验性策略）可以绑定不同的函数实现，实现策略级别的多态。C 语言没有内置的面向对象机制，函数指针是实现多态的常见方式。
- 解耦与维护性：调度核心只依赖抽象接口（`enqueue/dequeue/pick_next/...`），实现细节封装在具体的 `sched_class` 中，便于单独开发、测试和替换。
- 动态替换与运行时配置：在某些系统需求下，可以在运行时切换或注册新的调度类（实验/调优），函数指针使得替换变得轻量。
- 性能与清晰性：相比在核心调度路径中写大量 switch/if-else 分支，函数指针调用更清晰、代码更小且易于内联（编译器可在静态场景下内联具体实现）。

运行队列结构体 run_queue 的分析：

比较lab5和lab6中 run_queue 结构体的差异，解释为什么lab6的 run_queue 需要支持两种数据结构（链表和斜堆）：

Lab5中对调度的定义非常简洁，只声明了调度接口，没有run_queue定义或调度类抽象，实现通常只用一个链表或简单队列来放可运行进程。

```
1 void schedule(void);
2 void wakeup_proc(struct proc_struct *proc);
```

在Lab6中新增了struct sched_class，是一个可插拔的调度类接口，并把运行队列抽象为：

代码块

```
1 struct run_queue
2 {
3     list_entry_t run_list;
4     unsigned int proc_num;
5     int max_time_slice;
6     // For LAB6 ONLY
7     skew_heap_entry_t *lab6_run_pool;
8 };
```

其中list_entry_t run_list;为链表的定义，skew_heap_entry_t *lab6_run_pool;为斜堆的定义。

为什么lab6的 run_queue 需要支持两种数据结构（链表和斜堆）：

1. 支持多种调度策略（可扩展性）

- sched_class允许不同策略实现自己的 enqueue/dequeue/pick_next。
- 某些策略用链表最直观、实现最简单；而像基于优先级或 stride/比例调度需要快速选取“最优”进程，斜堆更合适。

2. 不同数据结构对应不同复杂度/语义

- 链表： $O(1)$ 的插入/删除（在已知位置），遍历找到 next 可能是 $O(n)$ 。适合轮转、队头队尾语义或当 pick_next 简单（例如按队头）时。
- 斜堆（优先队列）：支持 $O(\log n)$ 的插入与弹出最优元素，并且支持高效合并（meld），更适合需要按动态权重/优先级选择的调度（如最小虚拟运行时间、stride）。

3. 性能与语义权衡

- 当系统中进程数较大，且调度需要频繁按优先级做选择时，用堆能显著降低 pick_next 的代价。
- 对于简单/低开销策略，链表更节省空间与实现复杂度。

4. 向后兼容与演化路径

- 保留链表能让已有的简单策略继续工作；新增斜堆支持更复杂策略而不破坏现有代码。
- run_queue 中同时包含两者，调度类可按需使用其中之一或两者配合。

5. 斜堆的具体优势（相比二叉堆等）

- 斜堆实现简单、支持高效的 meld 操作，适合在多队列合并或迁移场景下使用。

调度器框架函数分析：

分析 sched_init()、wakeup_proc() 和 schedule() 函数在lab6中的实现变化

sched_init():

在Lab5中没有专门的sched_init()。

在Lab6中：

代码块

```
1 void sched_init(void) {
2     list_init(&timer_list);           // 初始化定时器链表
3     sched_class = &default_sched_class; // 选择默认调度类
4     rq = &__rq;                       // 指向静态运行队列
5     rq->max_time_slice = MAX_TIME_SLICE; // 设置时间片上限
6     sched_class->init(rq);             // 调用调度类的初始化
7     cprintf("sched class: %s\n", sched_class->name);
8 }
```

- 初始化timer列表；
- 选择默认的sched_class并把rq指向静态__rq；
- 为rq设定max_time_slice并调用sched_class->init(rq)做run-queue的初始化；
- 打印当前调度名。

把调度从“写死”的实现变成可配置/可扩展的框架，插件式地初始化不同策略需要的数据结构。

wakeup_proc:

在Lab5中仅把 `proc->state` 设为 `PROC_RUNNABLE`，清 `wait_state`，没有入队操作。

Lab6:

代码块

```
1 void wakeup_proc(struct proc_struct *proc) {
2     assert(proc->state != PROC_ZOMBIE);
3     bool intr_flag;
4     local_intr_save(intr_flag);
5     {
6         if (proc->state != PROC_RUNNABLE) {
7             proc->state = PROC_RUNNABLE; // 标记为可运行
8             proc->wait_state = 0;        // 清除等待状态
9             if (proc != current) {
10                 sched_class_enqueue(proc); // 立即放入运行队列
11             }
12         } else {
```

```
13         warn("wakeup runnable process.\n");
14     }
15 }
16 local_intr_restore(intr_flag);
17 }
```

1. 修改进程状态

- 操作: `proc->state=PROC_RUNNABLE`
- 含义: 把进程从“睡眠/等待”状态改为“可运行”状态
- 目的: 告诉系统“这个进程现在可以被调度了”

2. 清除等待原因

- 操作: `proc->wait_state=0`
- 含义: 清空之前记录的等待原因
- 目的: 表示该进程不再因任何原因等待

3. 决定是否入队

- 分为两种情况:
 - `proc!=current`
 - 执行: `sched_class_enqueue(proc)`
 - 含义: 调用`sched_class->enqueue(rq,proc)`, 把进程加入运行队列
 - 原因:
 - 这个进程刚被唤醒, 需要“排队等待CPU”
 - 把它放入`run_queue`, 下次`schedule()`时可能被选中运行
 - 具体怎么入队由`sched_class`决定
 - `proc==current`
 - 不执行入队
 - 原因:
 - 进程本来就在CPU上运行, 不需要排队
 - 它已经占有CPU资源, 无需放入等待队列

schedule:

在Lab5中线性遍历`proc_list`查找`PROC_RUNNABLE`。

Lab6:

```

1 void schedule(void)
2 {
3     bool intr_flag;
4     struct proc_struct *next;
5     local_intr_save(intr_flag);
6     {
7         current->need_resched = 0;
8         if (current->state == PROC_RUNNABLE)
9         {
10             sched_class_enqueue(current);
11         }
12         if ((next = sched_class_pick_next()) != NULL)
13         {
14             sched_class_dequeue(next);
15         }
16         if (next == NULL)
17         {
18             next = idleproc;
19         }
20         next->runs++;
21         if (next != current)
22         {
23             proc_run(next);
24         }
25     }
26     local_intr_restore(intr_flag);
27 }

```

- 三步走流程：
 - a. 当前进程入队（若仍可运行）
 - b. 选取下一个进程（`pick_next`）并出队
 - c. 切换到选中进程
- 策略无关：`schedule()`只管流程，具体选取算法由`sched_class->pick_next()`决定。
- idle保底：确保总有进程可运行。

理解这些函数如何与具体的调度算法解耦：

`sched_init()` 的解耦分析

代码块

```

1 void sched_init(void) {
2     list_init(&timer_list); // ① 框架初始化
3     sched_class = &default_sched_class; // ② 策略选择（解耦关键）

```

```

4      rq = &__rq; // ③ 队列绑定
5      rq->max_time_slice = MAX_TIME_SLICE; // ④ 通用配置
6      sched_class->init(rq); // ⑤ 策略初始化（解耦实现）
7      cprintf("sched class: %s\n", sched_class->name); // ⑥ 信息输出
8  }

```

1. list_init(&timer_list);-框架层逻辑

- 职责：初始化全局定时器链表
- 解耦特征：
 - 算法无关：无论用哪种调度算法，系统都需要定时器
 - 框架责任：由调度框架统一管理，不委派给具体算法
- 若耦合：

代码块

```

1  // 错误示例：把定时器初始化耦合到算法
2  if (use_rr_sched) {
3      list_init(&timer_list); // RR需要定时器
4  } else if (use_stride_sched) {
5      list_init(&timer_list); // Stride也需要
6  }
7  // 重复代码，违反DRY原则

```

2. sched_class = &default_sched_class;-核心解耦点

- 职责：选择具体的调度算法
- 解耦机制：

代码块

```

1  // 定义在 sched.c 中
2  static struct sched_class *sched_class; // 指针，可指向任何算法
3
4  // 可选的算法实现
5  extern struct sched_class default_sched_class; // 轮转调度
6  extern struct sched_class stride_sched_class; // Stride调度
7  extern struct sched_class priority_sched_class; // 优先级调度（假设）

```

- 切换算法只需一行：

代码块

```

1 // 方案A: 使用轮转调度
2 sched_class = &default_sched_class;
3
4 // 方案B: 使用Stride调度 (编译时或运行时切换)
5 // sched_class = &stride_sched_class;
6
7 // 方案C: 甚至可以运行时动态切换
8 if (system_load > 80%) {
9     sched_class = &stride_sched_class; // 高负载用Stride
10 } else {
11     sched_class = &default_sched_class; // 低负载用RR
12 }

```

- 解耦意义：
 - 无需修改 `sched_init()` 的其他代码
 - 无需重新编译整个内核
 - 支持模块化的算法开发

3. rq = &__rq;-队列绑定

- 职责：将运行队列指针指向静态分配的队列
- 解耦特征：

代码块

```

1 static struct run_queue __rq; // 静态队列实例

```

- run_queue结构体同时包含链表和斜堆：

代码块

```

1 struct run_queue {
2     list_entry_t run_list; // 给RR用
3     skew_heap_entry_t *lab6_run_pool; // 给Stride用
4     unsigned int proc_num;
5     int max_time_slice;
6 };

```

- 算法自主选择：RR用 `run_list`，Stride用 `lab6_run_pool`
- `sched_init()` 只负责提供容器，**不关心**容器里放什么

4. rq->max_time_slice = MAX_TIME_SLICE-通用配置

- 职责：设置时间片上限
- 解耦特征：
 - 策略参数：可能会被不同的算法使用
 - RR调度：直接使用这个值作为每个进程的时间片
 - Stride调度：作为时间片的参考上限

5. sched_class->init(rq); -委派初始化（解耦实现核心）

- 职责：让具体算法初始化自己需要的数据结构
- 解耦机制分析：
 - 接口定义(抽象契约)

代码块

```
1  struct sched_class {
2      const char *name;
3      void (*init)(struct run_queue *rq); // 初始化接口
4      // 其他接口...
5  };
```

- 轮转调度的实现：

代码块

```
1  // 文件: default_sched.c
2  static void RR_init(struct run_queue *rq) {
3      list_init(&(rq->run_list)); // 只初始化链表
4      rq->proc_num = 0;
5  }
6
7  struct sched_class default_sched_class = {
8      .name = "RR_scheduler",
9      .init = RR_init,
10     // ...
11 };
```

- Stride调度的实现：

代码块

```
1  // 文件: default_sched_stride.c
2  static void stride_init(struct run_queue *rq) {
3      list_init(&(rq->run_list));
4      rq->lab6_run_pool = NULL; // 初始化斜堆为空
```

```

5     rq->proc_num = 0;
6 }
7
8 struct sched_class stride_sched_class = {
9     .name = "stride_scheduler",
10    .init = stride_init,
11    // ...
12 };

```

■ 执行流程：

代码块

```

1  sched_init() 调用
2  ↓
3  sched_class->init(rq)
4  ↓
5  根据 sched_class 指向的对象，执行对应的 init 函数
6  └→ 若指向 default_sched_class: 执行 RR_init()
7  └→ 若指向 stride_sched_class: 执行 stride_init()

```

○ 解耦关键：

- sched_init() 不知道要初始化链表还是斜堆
- sched_init() 只关心"调用 init 接口"
- 具体做什么由 sched_class->init 的实现决定

解耦的完整架构图：

代码块

1			
2	sched_init() - 调度器初始化框架		
3			
4	① list_init(&timer_list)	[框架层]	
5	└→ 算法无关的系统初始化		
6			
7	② sched_class = &default_sched_class	[解耦关键]	
8	└→ 通过指针赋值选择算法，支持编译时/运行时切换		
9			
10	③ rq = &__rq	[框架层]	
11	└→ 提供统一的运行队列容器		
12			
13	④ rq->max_time_slice = ...	[配置层]	
14	└→ 通用参数，算法可选使用		



wakeup_proc:

```
代码块

1 void wakeup_proc(struct proc_struct *proc) {
2     assert(proc->state != PROC_ZOMBIE); // ⑥ 前置检查
3     bool intr_flag;
4     local_intr_save(intr_flag); // ① 关中断保护
5     {
6         if (proc->state != PROC_RUNNABLE) {
7             proc->state = PROC_RUNNABLE; // ② 状态转换 (框架层)
8             proc->wait_state = 0; // ③ 清除等待原因 (框架层)
9             if (proc != current) { // ④ 条件判断 (框架层)
10                sched_class_enqueue(proc); // ⑤ 入队操作 (解耦核心)
11            }
12        } else {
```

```
13         warn("wakeup runnable process.\n");
14     }
15 }
16 local_intr_restore(intr_flag);           // ⑥ 恢复中断
17 }
```

我们先对其中的代码进行职责划分：



其中解耦的核心机制为**sched_class_enqueue()**的间接层，其实现代码为：

```

1 static inline void
代码块
2 sched_class_enqueue(struct proc_struct *proc)
3 {
4     if (proc != idleproc) // 框架层额外判断
5     {
6         sched_class->enqueue(rq, proc); // ★ 多态调用
7     }
8 }

```

解耦分析：

代码块

```

1  wakeup_proc() 调用栈的三层结构
2  =====
3
4  第1层：调用者层 (wakeup_proc)
5  -----
6  • 只知道"需要唤醒进程"
7  • 不关心如何入队
8  • 不依赖具体算法
9
10         |
11         | 调用
12         ↓
13
14  第2层：抽象层 (sched_class_enqueue)
15  -----
16  • 提供统一接口
17  • 隐藏算法细节
18  • 通过函数指针转发
19
20         |
21         | sched_class->enqueue(rq, proc)
22         ↓
23
24  第3层：算法层 (RR_enqueue / stride_enqueue)
25  -----
26  • 实现具体策略
27  • 独立可替换
28  • 无需通知上层

```

最后完成多态调用的实现机制。运行时多态展开：

代码块

```

1 // 在 wakeup_proc() 中的调用
2 sched_class_enqueue(proc);
3   ↓ [展开内联函数]
4 sched_class->enqueue(rq, proc);
5   ↓ [查找函数指针表]
6
7 | sched_class 指向的对象 |
8 |
9 | 如果是 default_sched_class: |
10 |   → RR_enqueue(rq, proc) |
11 |
12 | 如果是 stride_sched_class: |
13 |   → stride_enqueue(rq, proc) |
14 |

```

我们可以通过展示运行时内存布局来对解耦的优势进行阐述：

代码块

```

1 运行时内存状态
2  =====
3
4 全局变量区:
5
6 | sched_class (指针变量) | ← wakeup_proc 使用这个指针
7 | |
8 | | 指向
9 | | ↓
10 |
11 | default_sched_class (对象) |
12 | |
13 | | .name = "RR_scheduler" | |
14 | | .init = RR_init | |
15 | | .enqueue = RR_enqueue | ← 调用这个函数
16 | | .dequeue = RR_dequeue | |
17 | | .pick_next = RR_pick_next | |
18 | | .proc_tick = RR_proc_tick | |
19 | |
20 |
21
22 代码区:
23
24 | RR_enqueue() {
25 |     list_add_before(...); | ← 实际执行的代码
26 |     proc->time_slice = ...; |
27 |     rq->proc_num++; |

```

```
28 | }
29 |_____
```

根据这一段可知，我们只需在初始化时将name值初始化为对应的调度算法，之后，就不需要对代码进行修改，如果想要修改使用的调度算法，只需修改name的值即可，其他的代码的内容均不需要进行改动。

schedule:

解耦前，也就是如果调度逻辑直接写死在schedule()里：

代码块

```
1 void schedule(void) {
2     // 写死的轮转调度逻辑
3     struct proc_struct *next = NULL;
4     list_entry_t *le = list_next(&(current->list_link));
5
6     while (le != &proc_list) {
7         next = le2proc(le, list_link);
8         if (next->state == PROC_RUNNABLE) {
9             break; // 找到第一个可运行的
10        }
11        le = list_next(le);
12    }
13
14    proc_run(next);
15 }
```

问题：

- 想换成优先级调度就必须重写整个schedule()
- 如果想支持多种算法，代码会充满if-else判断
- schedule()既管流程又管算法，职责混乱

解耦后，通过sched_class接口分离：

代码块

```
1 // schedule() 只管"流程"
2 void schedule(void) {
3     if (current->state == PROC_RUNNABLE) {
4         sched_class_enqueue(current); // 调用接口，不管怎么入队
5     }
6
7     next = sched_class_pick_next(); // 调用接口，不管怎么选
```

```

8
9     if (next != NULL) {
10         sched_class_dequeue(next);           // 调用接口，不管怎么出队
11     }
12
13     proc_run(next);
14 }
15
16 // 具体算法在 sched_class 实现里
17 struct sched_class {
18     void (*enqueue)(struct run_queue *rq, struct proc_struct *proc);
19     void (*dequeue)(struct run_queue *rq, struct proc_struct *proc);
20     struct proc_struct *(*pick_next)(struct run_queue *rq);
21     // ...
22 };

```

其中，解耦实现了：

1. 职责分离：

模块	职责	不关心
schedule()	调度流程：何时入队、何时选取、何时切换	用什么数据结构、怎么选
sched_class	调度策略：怎么入队、选谁、用什么结构	何时被调用、进程切换细节

2. 接口与实现分离

代码块

```

1 // 接口定义 (sched.h)
2 struct sched_class {
3     struct proc_struct *(*pick_next)(struct run_queue *rq);
4 };
5
6 // 实现1: 轮转调度 (default_sched.c)
7 static struct proc_struct *RR_pick_next(struct run_queue *rq) {
8     list_entry_t *le = list_next(&(rq->run_list));
9     return le2proc(le, run_link); // 返回链表头
10 }
11
12 // 实现2: Stride调度 (default_sched_stride.c)
13 static struct proc_struct *stride_pick_next(struct run_queue *rq) {
14     skew_heap_entry_t *le = skew_heap_min(rq->lab6_run_pool);

```



```

15     return le2proc(le, lab6_run_pool); // 返回堆顶 (最小stride)
16 }

```

关键：schedule()只调用sched_class->pick_next(), 不知道也不需要知道是链表还是堆。

3. 可替换性

切换调度算法修改一行：

代码块

```

1 void sched_init(void) {
2     // 方案A: 使用轮转
3     sched_class = &default_sched_class;
4
5     // 方案B: 使用Stride (只需改这一行)
6     // sched_class = &stride_sched_class;
7
8     sched_class->init(rq); // 后续代码不变
9 }

```

问题解答：

1.调度类的初始化流程：描述从内核启动到调度器初始化完成的完整流程，分析default_sched_class 如何与调度器框架关联。

汇编入口

代码块

```

1 kern_entry:
2     # 1. 保存硬件线程ID和设备树地址
3     la t0, boot_hartid
4     sd a0, 0(t0)
5
6     # 2. 设置页表, 启用虚拟内存
7     lui t0, %hi(boot_page_table_sv39)
8     # ... 页表配置 ...
9     csrw satp, t0          # 写入 satp 寄存器
10    sfence.vma             # 刷新 TLB
11
12    # 3. 设置内核栈
13    lui sp, %hi(bootstacktop)
14
15    # 4. 跳转到 C 代码入口
16    lui t0, %hi(kern_init)

```

```
17     addi t0, t0, %lo(kern_init)
18     jr t0                # 跳转到 kern_init
```

关键点：

- 建立虚拟内存映射
- 设置内核栈为虚拟地址
- 跳转到C语言环境

内核初始化

代码块

```
1  int kern_init(void) {
2      // 1. 清空 BSS 段
3      memset(edata, 0, end - edata);
4
5      // 2. 基础设施初始化
6      cons_init();        // 控制台
7      print_kerninfo();
8      dtb_init();         // 设备树
9
10     // 3. 内存管理初始化
11     pmm_init();          // 物理内存管理
12     vmm_init();          // 虚拟内存管理
13
14     // 4. 中断系统初始化
15     pic_init();          // 中断控制器
16     idt_init();          // 中断描述符表
17
18     // ★ 5. 调度器初始化 (关键步骤)
19     sched_init();
20
21     // 6. 进程系统初始化
22     proc_init();
23
24     // 7. 时钟中断初始化
25     clock_init();
26     intr_enable();       // 开启中断
27
28     // 8. 进入空闲进程循环
29     cpu_idle();          // 永不返回
30 }
```

调用顺序：sched_init()在proc_init()之前，确保调度器框架就绪后再创建进程。

调度器初始化详解

在sched.c中的实现

代码块

```
1  static struct sched_class *sched_class; // 调度算法指针 (全局)
2  static struct run_queue *rq;           // 运行队列指针 (全局)
3  static struct run_queue __rq;          // 实际的运行队列对象
4
5  void sched_init(void) {
6      // 1. 初始化定时器链表
7      list_init(&timer_list);
8
9      // ★ 2. 关联调度算法类 (核心步骤)
10     sched_class = &default_sched_class;
11
12     // 3. 关联运行队列
13     rq = &__rq;
14     rq->max_time_slice = MAX_TIME_SLICE;
15
16     // ★ 4. 调用算法的初始化函数 (多态调用)
17     sched_class->init(rq);
18
19     // 5. 打印调度器信息
20     cprintf("sched class: %s\n", sched_class->name);
21 }
```

default_sched_class与框架的关联机制

1.default_sched_class的定义

代码块

```
1  struct sched_class default_sched_class = {
2      .name      = "RR_scheduler", // 名称: 轮转调度器
3      .init      = RR_init,        // 初始化函数
4      .enqueue   = RR_enqueue,     // 入队函数
5      .dequeue   = RR_dequeue,     // 出队函数
6      .pick_next = RR_pick_next,   // 选择下一个进程
7      .proc_tick = RR_proc_tick,   // 时钟中断处理
8  };
```

这是一个静态全局对象，在编译时就已存在于内核镜像中。

2.关联过程的详细分析

步骤1：指针赋值（绑定算法）

代码块

```
1 sched_class = &default_sched_class;
```

内存布局：

代码块

```
1 内核镜像 .data 段：
2
3 | default_sched_class (全局对象) |
4 | .name      → "RR_scheduler"  |
5 | .init      → RR_init         |
6 | .enqueue   → RR_enqueue      |
7 | .dequeue   → RR_dequeue      |
8 | .pick_next → RR_pick_next    |
9 | .proc_tick → RR_proc_tick    |
10 |
11      ↑
12      | 指针指向
13      |
14 | sched_class | (全局指针变量)
15 |
16
```

步骤2：多态调用算法初始化

代码块

```
1 sched_class->init(rq);
```

等价于：

代码块

```
1 default_sched_class.init(rq);
2 // 即调用 RR_init(rq);
```

RR_init的实现（轮转调度算法初始化）：

代码块

```

1  static void RR_init(struct run_queue *rq) {
2      list_init(&(rq->run_list));    // 初始化链表
3      rq->proc_num = 0;              // 进程数清零
4  }

```

完整时序图

代码块

1 时间轴:

2 =====

3 =====

4 [T0] entry.S: kern_entry

5 |— 设置页表

6 |— 设置栈

7 |— jr kern_init

8

9 [T1] init.c: kern_init()

10 |— cons_init()

11 |— pmm_init()

12 |— vmm_init()

13 |— pic_init()

14 |— idt_init()

15 |

16 |— ★ sched_init() ← 调度器初始化

17 | |— list_init(&timer_list)

18 | |— sched_class = &default_sched_class ← 绑定算法

19 | |— rq = &__rq

20 | |— sched_class->init(rq) ← 调用 RR_init()

21 | |— list_init(&(rq->run_list))

22 |

23 |— proc_init() ← 进程初始化

24 | |— 创建 idleproc (pid=0)

25 | |— 创建 initproc (pid=1, 调用 wakeup_proc)

26 | |— wakeup_proc()

27 | |— sched_class->enqueue() ← 使用调度器

28 |

29 |— clock_init()

30 |— intr_enable()

31 |

32 |— cpu_idle() ← 进入调度循环

33 | |— while(1) { schedule(); }

34 | |— sched_class->pick_next() ← 持续使用调度器

35

2.进程调度流程：绘制一个完整的进程调度流程图

流程图如下：

代码块		时间	事件	标志状态	队列状态
1		=====	=====	=====	=====
2					
3					
4	T=0		进程A运行 (time_slice=5)	need_resched=0	rq: [B, C]
5					
6					
7	T=10ms		时钟中断触发		
8			clock_set_next_event()		
9			ticks++		
10			RR_proc_tick(A)		
11			A->time_slice-- (5→4)		
12			未用完，不设置标志		
13			返回 trap()		
14			检查 need_resched=0		
15			不调度，继续运行A	need_resched=0	rq: [B, C]
16					
17	T=20ms		时钟中断触发		
18			RR_proc_tick(A)		
19			A->time_slice-- (4→3)	need_resched=0	rq: [B, C]
20			继续运行A		
21					
22	T=30ms		时钟中断触发		
23			RR_proc_tick(A)		
24			A->time_slice-- (3→2)	need_resched=0	rq: [B, C]
25			继续运行A		
26					
27	T=40ms		时钟中断触发		
28			RR_proc_tick(A)		
29			A->time_slice-- (2→1)	need_resched=0	rq: [B, C]
30			继续运行A		
31					
32	T=50ms		时钟中断触发 ★ 关键时刻		
33			RR_proc_tick(A)		
34			A->time_slice-- (1→0)		
35			time_slice==0		
36			A->need_resched = 1 ★	need_resched=1	rq: [B, C]
37					
38			trap() 返回前		

```

39         └─ if (A->need_resched)
40             |
41             └─ schedule() ★ 开始调度
42                 |
43                 └─ A->need_resched = 0          need_resched=0 [清除标志]
44                     |
45                     └─ sched_class_enqueue(A)    -          rq: [B, C, A]
46                         └─ A->time_slice = 5 (重置)      [A移到队尾]
47                             |
48                             └─ next = pick_next()    -          [选中B]
49                                 └─ 返回 B
50                                     |
51                                     └─ sched_class_dequeue(B) -          rq: [C, A]
52                                         [B移出队列]
53                                             |
54                                             └─ proc_run(B) ★ 切换到B
55                                                 |
56                                                 └─ 切换页表
57                                                     |
58                                                     └─ 切换栈
59                                                         |
60                                                         └─ switch_to(A, B)

```

59 T=51ms 进程B开始运行 (time_slice=5) B->need_resched=0 rq: [C, A]

3.调度算法的切换机制：分析如果要添加一个新的调度算法（如stride），需要修改哪些代码？并解释为什么当前的设计使得切换调度算法变得容易。

步骤一：实现调度算法类

创建新文件：kern/schedule/my_new_sched.c

代码块

```

1  #include <defs.h>
2  #include <list.h>
3  #include <proc.h>
4  #include <assert.h>
5
6  // -----
7  // 步骤 1.1: 实现 init 函数
8  // -----
9  static void my_sched_init(struct run_queue *rq) {
10     list_init(&(rq->run_list)); // 初始化队列
11     rq->proc_num = 0;           // 进程数清零
12     // 如果需要额外数据结构，在这里初始化
13 }
14
15 // -----

```

```

16 // 步骤 1.2: 实现 enqueue 函数
17 // -----
18 static void my_sched_enqueue(struct run_queue *rq, struct proc_struct *proc) {
19     assert(list_empty(&(amp;proc->run_link)));
20
21     // 实现你的入队策略 (例如: 按优先级插入)
22     list_add_before(&(amp;rq->run_list), &(proc->run_link));
23
24     // 设置时间片
25     if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice) {
26         proc->time_slice = rq->max_time_slice;
27     }
28
29     proc->rq = rq;
30     rq->proc_num++;
31 }
32
33 // -----
34 // 步骤 1.3: 实现 dequeue 函数
35 // -----
36 static void my_sched_dequeue(struct run_queue *rq, struct proc_struct *proc) {
37     assert(!list_empty(&(amp;proc->run_link)) && proc->rq == rq);
38
39     list_del_init(&(amp;proc->run_link)); // 从队列中移除
40     rq->proc_num--;
41 }
42
43 // -----
44 // 步骤 1.4: 实现 pick_next 函数
45 // -----
46 static struct proc_struct *my_sched_pick_next(struct run_queue *rq) {
47     if (list_empty(&(amp;rq->run_list))) {
48         return NULL;
49     }
50
51     // 实现你的选择策略 (例如: 选择优先级最高的)
52     list_entry_t *le = list_next(&(amp;rq->run_list));
53
54     // 可以遍历找最优进程
55     // list_entry_t *best = le;
56     // while ((le = list_next(le)) != &(rq->run_list)) {
57     //     // 比较逻辑
58     // }
59
60     return le2proc(le, run_link);
61 }
62

```



```

63 // -----
64 // 步骤 1.5: 实现 proc_tick 函数
65 // -----
66 static void my_sched_proc_tick(struct run_queue *rq, struct proc_struct *proc)
67 {
68     if (proc->time_slice > 0) {
69         proc->time_slice--;
70     }
71     if (proc->time_slice == 0) {
72         // 时间片用完, 设置调度标志
73         proc->need_resched = 1;
74     }
75 }
76 // -----
77 // ★ 步骤 1.6: 定义调度类对象 (关键)
78 // -----
79 struct sched_class my_new_sched_class = {
80     .name      = "my_new_scheduler", // 调度器名称
81     .init      = my_sched_init,      // 函数指针绑定
82     .enqueue   = my_sched_enqueue,
83     .dequeue   = my_sched_dequeue,
84     .pick_next = my_sched_pick_next,
85     .proc_tick = my_sched_proc_tick,
86 };

```

步骤二：在头文件中声明

修改kern/schedule/default_sched.h:

代码块

```

1  #ifndef __KERN_SCHEDULE_SCHED_RR_H__
2  #define __KERN_SCHEDULE_SCHED_RR_H__
3
4  #include <sched.h>
5
6  extern struct sched_class default_sched_class; // RR 调度器
7  extern struct sched_class stride_sched_class; // Stride 调度器
8  extern struct sched_class my_new_sched_class; // ★ 新增: 你的调度器
9
10 #endif

```

步骤三：在调度框架中切换算法

修改kern/schedule/sched.c的sched_init()函数:

代码块

```

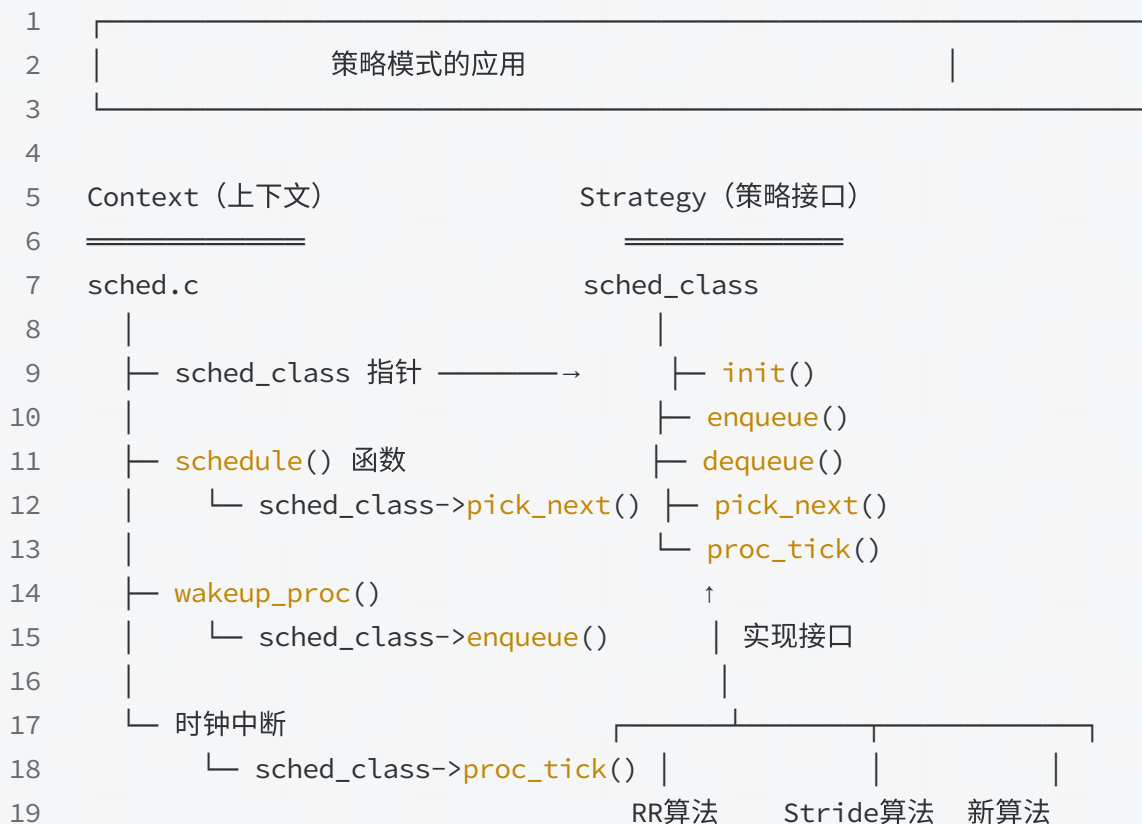
1  #include <default_sched.h> // 包含调度器声明
2
3  void sched_init(void) {
4      list_init(&timer_list);
5
6      // ★ 方式1: 直接修改这一行即可切换调度器
7      // sched_class = &default_sched_class; // 使用 RR 调度器
8      // sched_class = &stride_sched_class; // 使用 Stride 调度器
9      sched_class = &my_new_sched_class; // 使用新调度器 ★
10
11     rq = &__rq;
12     rq->max_time_slice = MAX_TIME_SLICE;
13     sched_class->init(rq); // 自动调用对应的 init 函数
14
15     cprintf("sched class: %s\n", sched_class->name);
16 }

```

为什么切换调度算法如此容易？

设计原理1：策略模式

代码块



关键机制：

代码块

```
1 // 框架层不关心具体算法，只调用接口
2 void schedule(void) {
3     // 使用哪个算法？由 sched_class 指针决定
4     next = sched_class->pick_next(rq); // 多态调用
5 }
6
7 // 运行时动态绑定
8 sched_class = &my_new_sched_class; // 切换算法
9 // 之后所有调用自动使用新算法
```

设计原理2：依赖倒置原则

代码块

1 传统设计（耦合）	现代设计（解耦）
2 <u> </u>	2 <u> </u>
3	
4 schedule() {	4 schedule() {
5 // 硬编码算法	5 // 依赖抽象接口
6 if (使用RR) {	6 sched_class->pick_next();
7 RR_pick_next();	7 ↑
8 } else if (使用Stride) {	8 只依赖接口
9 Stride_pick_next();	9 不依赖具体实现
10 }	10
11 // 每增加一个算法	11
12 // 都要修改这里	12
13 }	13 RR Stride 新算法
14	14 ↑
15	15 └─ 具体算法实现接口

依赖关系：

代码块

1 高层模块		2 底层模块
2 <u> </u>		2 <u> </u>
3 schedule()	← 依赖 —	抽象接口 (sched_class)
4 wakeup_proc()	← 依赖 —	↑
5 trap.c	← 依赖 —	实现
6		
7		├──────────┤
8		
9		RR算法 Stride算法
10		

设计原理3：接口统一性

代码块

```

1  所有调度算法必须实现相同的接口
2  =====
3
4  struct sched_class {
5      const char *name;
6      void (*init)(struct run_queue *rq);
7      void (*enqueue)(struct run_queue *rq, struct proc_struct *proc);
8      void (*dequeue)(struct run_queue *rq, struct proc_struct *proc);
9      struct proc_struct *(*pick_next)(struct run_queue *rq);
10     void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc);
11 };
12
13
14 | 无论内部实现多么不同，外部接口完全一致 |
15 |-----|
16
17 RR算法:           Stride算法:           新算法:
18 |— 链表实现        |— 斜堆实现            |— 红黑树?
19 |— O(1)入队        |— O(log n)入队        |— O(?)入队
20 |— 时间片轮转      |— 步长调度            |— 自定义策略
21   ↓                ↓                    ↓
22   └────────────────┴────────────────┴────────────────┘
23   |                 |                 |
24   [相同的接口签名]  [相同的调用方式]

```

二、练习2：实现 Round Robin 调度算法

问题分析

1. Lab5与Lab6函数实现对比分析

选择函数： `schedule()` 函数

Lab5中的实现：

代码块

```

1  void schedule(void) {
2      bool intr_flag;

```

```

3     list_entry_t *le, *last;
4     struct proc_struct *next = NULL;
5     local_intr_save(intr_flag);
6     {
7         current->need_resched = 0;
8         last = (current == idleproc) ? &proc_list : &(current->list_link);
9         le = last;
10        do {
11            if ((le = list_next(le)) != &proc_list) {
12                next = le2proc(le, list_link);
13                if (next->state == PROC_RUNNABLE) {
14                    break;
15                }
16            }
17        } while (le != last);
18        if (next == NULL || next->state != PROC_RUNNABLE) {
19            next = idleproc;
20        }
21        next->runs++;
22        if (next != current) {
23            proc_run(next);
24        }
25    }
26    local_intr_restore(intr_flag);
27 }

```

Lab6中的实现：

代码块

```

1     void schedule(void) {
2         bool intr_flag;
3         struct proc_struct *next;
4         local_intr_save(intr_flag);
5         {
6             current->need_resched = 0;
7             if (current->state == PROC_RUNNABLE) {
8                 sched_class_enqueue(current);
9             }
10            if ((next = sched_class_pick_next()) != NULL) {
11                sched_class_dequeue(next);
12            }
13            if (next == NULL) {
14                next = idleproc;
15            }
16            next->runs++;

```

```

17         if (next != current) {
18             proc_run(next);
19         }
20     }
21     local_intr_restore(intr_flag);
22 }

```

为什么要做这个改动：

改动原因：

1. **架构层面的提升**：Lab5中调度逻辑直接写死在schedule()函数中，通过遍历全局进程链表proc_list来查找下一个可运行的进程。Lab6引入了调度器框架（sched_class），将调度策略从调度流程中分离，实现了策略与机制的解耦。
2. **使用专用运行队列**：Lab5使用全局进程链表proc_list（包含所有状态的进程），而Lab6引入了专用的run_queue（仅包含可运行进程），避免了遍历睡眠/僵尸进程的开销。
3. **支持多种调度策略**：Lab6通过函数指针机制（sched_class->enqueue/dequeue/pick_next），可以在运行时或编译时切换不同的调度算法（RR、Stride等），而Lab5的硬编码方式无法灵活切换。
4. **性能优化**：
 - Lab5：O(n)遍历所有进程，效率低下
 - Lab6：调度类可以选择高效的数据结构（链表O(1)、斜堆O(log n)），显著提升性能

不做这个改动会出现的问题：

1. **扩展性差**：添加新调度算法需要大量修改核心调度代码，容易引入bug
2. **性能低下**：遍历proc_list会包含所有非RUNNABLE进程，浪费CPU时间
3. **维护困难**：调度策略与调度框架耦合，代码难以维护和测试
4. **功能受限**：无法支持需要特殊数据结构的调度算法（如优先队列、红黑树等）

函数实现详解

1. RR_init 函数

实现代码：

代码块

```

1  static void
2  RR_init(struct run_queue *rq)
3  {
4      // LAB6: 2312130
5      list_init(&(rq->run_list)); // 初始化就绪队列链表头(空队列)
6      rq->proc_num = 0;           // 当前就绪进程数为0

```

```
7 }
```

具体思路和方法：

- **功能：**初始化运行队列的基本结构
- **实现步骤：**
 - a. 使用 `list_init` 初始化循环双向链表头，使其指向自己（空队列状态）
 - b. 将进程计数器 `proc_num` 置为0
 - c. `max_time_slice` 由调用者（`sched_init`）设置，无需在此处理
- **链表操作选择：**使用 `list_init` 而非手动设置prev/next，利用已有接口保证正确性
- **边界条件处理：**初始化后的空队列状态为 `run_list.prev == run_list.next == &run_list`

2. RR_enqueue 函数

实现代码：

代码块

```
1  static void
2  RR_enqueue(struct run_queue *rq, struct proc_struct *proc)
3  {
4      // LAB6: 2312130
5      assert(rq != NULL && proc != NULL); // 基本健壮性检查
6      assert(list_empty(&(proc->run_link))); // 关键：防止重复入队破坏链表
7
8      proc->rq = rq; // 记录该进程所在的 run_queue
9      if (proc->time_slice <= 0)
10     { // 时间片用尽/新建进程，需重新分配
11         proc->time_slice = rq->max_time_slice; // 统一设置为最大时间片
12     }
13
14     // RR: 入队到队尾(链表头 run_list 的前一个位置即队尾)
15     list_add_before(&(rq->run_list), &(proc->run_link)); // 将进程挂到队尾
16     rq->proc_num++; // 更新就绪队列进程数
17 }
```

具体思路和方法：

- **功能：**将进程插入运行队列的尾部（FIFO原则）
- **实现步骤：**

- a. **健壮性检查**: 确保rq和proc非空, 且进程未在其他队列中 (`list_empty` 检查)
- b. **建立关联**: 设置 `proc->rq` 指针, 记录进程所属队列
- c. **时间片管理**:
 - 如果 `time_slice <= 0` (新建进程或时间片耗尽), 重新分配完整时间片
 - 否则保留剩余时间片 (支持抢占后恢复)
- d. **插入队尾**: 使用 `list_add_before` 在链表头前插入 (即队尾), 实现FIFO
- e. **更新元数据**: 递增 `proc_num`
- **链表操作选择**:
 - 使用 `list_add_before(&run_list, &run_link)` 而非 `list_add_after`, 因为循环链表中头节点的前驱是尾部
 - 队列结构: `run_list -> 队首 -> ... -> 队尾 -> run_list`
- **边界条件处理**:
 - **空队列**: `list_add_before` 正确处理, 插入后成为唯一元素
 - **重复入队**: 通过 `assert(list_empty(&proc->run_link))` 捕获错误
 - **时间片异常**: 通过 `if (proc->time_slice <= 0)` 统一处理

3. RR_dequeue 函数

实现代码:

代码块

```

1  static void
2  RR_dequeue(struct run_queue *rq, struct proc_struct *proc)
3  {
4      // LAB6: 2312130
5      assert(rq != NULL && proc != NULL); // 基本健壮性检查
6      assert(proc->rq == rq);
7      list_del_init(&(proc->run_link)); // 从就绪队列中摘除并重新初始化结点
8      rq->proc_num--;                  // 更新就绪队列进程数
9  }
```

具体思路和方法:

- **功能**: 将进程从运行队列中移除
- **实现步骤**:
 - a. **健壮性检查**: 确保进程确实属于该队列 (`proc->rq == rq`)

b. 摘除节点：使用 `list_del_init` 从链表中移除并重新初始化节点

c. 更新元数据：递减 `proc_num`

- 链表操作选择：

- 使用 `list_del_init` 而非 `list_del`，因为 `init` 会将节点的prev/next指向自己
- 重新初始化后可以用 `list_empty` 检查，防止野指针和重复操作

- 边界条件处理：

- 队列变空：`list_del_init` 正确处理最后一个元素的移除
- 进程不在队列中：通过 `assert(proc->rq == rq)` 捕获错误
- `proc_num`不会下溢：前置断言保证了进程在队列中

4. RR_pick_next 函数

实现代码：

代码块

```
1  static struct proc_struct *
2  RR_pick_next(struct run_queue *rq)
3  {
4      // LAB6: 2312130
5      assert(rq != NULL); // run_queue 必须存在
6
7      if (list_empty(&(rq->run_list)))
8      { // 队列为空，无可运行进程
9          return NULL;
10     }
11     list_entry_t *le = list_next(&(rq->run_list)); // 取队头元素(链表头的下一个)
12     return le2proc(le, run_link); // 由链表结点反推出
13     proc_struct
14 }
```

具体思路和方法：

- 功能：选择下一个要运行的进程（队首元素）
- 实现步骤：
 - a. 空队列检查：如果队列为空，返回NULL（调用者会选择idleproc）
 - b. 获取队首：使用 `list_next` 取链表头的下一个节点（即队首）
 - c. 类型转换：使用 `le2proc` 宏从 `run_link` 字段反推出完整的 `proc_struct` 指针
- 链表操作选择：

- 使用 `list_next(&run_list)` 取队首，而非遍历查找
- FIFO策略：总是选择最早入队的进程
- 宏的使用：
 - `le2proc(le, run_link)` 等价于 `to_struct(le, struct proc_struct, run_link)`
 - 通过成员指针偏移计算结构体首地址： `(type *)((char *) (ptr) - offsetof(type, member))`
- 边界条件处理：
 - 空队列：返回NULL，由schedule()处理（选择idleproc）
 - 非空队列：始终能正确返回队首进程

5. RR_proc_tick 函数

实现代码：

代码块

```

1  static void
2  RR_proc_tick(struct run_queue *rq, struct proc_struct *proc)
3  {
4      // LAB6: 2312130
5      assert(rq != NULL && proc != NULL); // 基本健壮性检查
6      assert(proc->rq == rq);
7      if (proc->time_slice > 0)
8      {
9          // 仍有剩余时间片
10         proc->time_slice--; // 每次时钟中断消耗一个时间片
11     }
12     if (proc->time_slice <= 0)
13     { // 时间片耗尽，需要触发调度
14         proc->need_resched = 1;
15     } // 设置重调度标志，trap 返回前将调用 schedule()
16 }
```

具体思路和方法：

- 功能：处理每个时钟tick，维护当前进程的时间片
- 实现步骤：
 - 时间片递减**：如果 `time_slice > 0`，递减1（表示消耗一个时间片单位）
 - 调度触发**：如果时间片耗尽（`<= 0`），设置 `need_resched = 1`
 - 延迟调度**：不立即调用schedule()，而是设置标志，由trap返回路径统一处理

- 边界条件处理：
 - 时间片为0：不会继续递减（避免负数溢出）
 - 时间片已负：通过 `<= 0` 判断统一处理（兼容异常情况）
 - 空闲进程：由 `sched_class_proc_tick` 在调用前过滤（idleproc直接设置 `need_resched`）
- 为什么需要设置`need_resched`标志：
 - a. 中断上下文限制：proc_tick在时钟中断处理中调用，不应直接进行进程切换
 - b. 统一调度点：所有调度请求（时间片耗尽/主动让出/阻塞唤醒）都通过`need_resched`触发
 - c. 原子性保证：trap返回前检查`need_resched`，在中断关闭时统一调度，避免竞态
 - d. 延迟处理：允许在中断返回前完成其他必要操作（如信号处理、抢占检查）

测试结果

make grade 输出结果：

```
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 1
set priority to 2
set priority to 3
set priority to 4
set priority to 5
child pid 3, acc 608000, time 2010
child pid 4, acc 612000, time 2010
child pid 5, acc 616000, time 2010
child pid 6, acc 580000, time 2010
child pid 7, acc 544000, time 2020
main: pid 0, acc 608000, time 2020
main: pid 4, acc 612000, time 2020
main: pid 5, acc 616000, time 2020
main: pid 6, acc 580000, time 2020
main: pid 7, acc 544000, time 2020
main: wait pids over
sched result: 1 1 1 1 1
```

QEMU中观察到的调度现象：

1. 时间片轮转行为：
 - 多个进程按FIFO顺序依次获得CPU时间片
 - 每个进程执行MAX_TIME_SLICE（默认值）个tick后被切换
2. 进程切换频率：
 - 可以通过观察进程的runs计数器增长，确认每个进程都得到公平调度

- 优先级相同的进程获得相等的CPU时间

3. 空闲进程行为：

- 当没有其他进程就绪时，idleproc运行
- idleproc的need_resched始终为1，一旦有进程就绪立即切换

4. 进程同步现象：

- 在forktest等测试中，父子进程交替执行
- wait/exit系统调用正确触发进程状态转换和调度

Round Robin 算法分析

1. 算法优缺点

优点：

1. **公平性好**：所有进程获得相等的CPU时间片，避免饥饿
2. **实现简单**：使用FIFO队列，算法逻辑清晰，易于理解和实现
3. **响应时间可预测**：最大等待时间 = (进程数 - 1) × 时间片长度
4. **适合分时系统**：对交互式任务友好，提供良好的响应性
5. **开销低**：O(1)的入队/出队/选择操作，无需复杂计算

缺点：

1. **不区分优先级**：所有进程平等对待，无法保证重要任务优先执行
2. **无法区分I/O密集与CPU密集**：对不同特性的任务一视同仁
3. **时间片大小敏感**：
 - 太大：退化为FCFS，响应时间变长
 - 太小：上下文切换开销占比过高
4. **平均周转时间较长**：短作业可能需要等待多轮才能完成
5. **不适合实时系统**：无法提供硬实时保证

2. 时间片大小调整策略

性能影响分析：

- **时间片 → 0**：频繁切换，CPU用于上下文保存/恢复，吞吐量下降
- **时间片 → ∞**：变为非抢占式FCFS，交互响应性差

优化：

1. **经验法则：**时间片应略大于典型上下文切换时间的10-100倍

2. **动态调整：**

- 根据系统负载自适应调整
- I/O密集进程：较小时间片（频繁主动让出）
- CPU密集进程：较大时间片（减少切换开销）

3. **混合策略：**

- 多级反馈队列：不同优先级使用不同时间片
- 短进程优先：首次分配小时间片，逐步增大

3. need_resched 标志的必要性

为什么需要设置该标志：

1. **中断上下文安全：**

- 时钟中断处理中不能直接调用schedule()（可能持有锁、栈空间有限）
- need_resched作为"延迟调度请求"，在安全点（trap返回前）执行

2. **调度时机统一：**

- 主动让出（do_yield）
- 时间片耗尽（proc_tick）
- 进程阻塞/唤醒（wait/wakeup）
- 所有路径通过need_resched统一触发，逻辑清晰

3. **避免嵌套调度：**

- 防止在schedule()执行过程中再次被调度（递归调用）
- 中断嵌套时保证调度的原子性

4. **性能优化：**

- 可以在中断返回前批处理多个调度请求
- 避免不必要的立即切换（如进程即将阻塞）

1. 实现优先级RR调度

需要修改的部分：

数据结构修改：

代码块

```
1 struct run_queue {
```

```

2     list_entry_t run_list[MAX_PRIORITY]; // 多个优先级队列
3     unsigned int proc_num;
4     int max_time_slice;
5 };

```

RR_enqueue 修改:

代码块

```

1 static void RR_enqueue(struct run_queue *rq, struct proc_struct *proc) {
2     int priority = proc->lab6_priority; // 获取进程优先级
3     list_add_before(&(rq->run_list[priority]), &(proc->run_link));
4     rq->proc_num++;
5 }

```

RR_pick_next 修改:

代码块

```

1 static struct proc_struct *RR_pick_next(struct run_queue *rq) {
2     for (int i = MAX_PRIORITY - 1; i >= 0; i--) { // 从高优先级开始
3         if (!list_empty(&(rq->run_list[i]))) {
4             list_entry_t *le = list_next(&(rq->run_list[i]));
5             return le2proc(le, run_link);
6         }
7     }
8     return NULL;
9 }

```

优先级动态调整:

- 可选: 实现优先级老化机制, 防止低优先级进程饥饿
- 长时间等待的进程逐步提升优先级

2. 多核调度支持

当前实现的局限性:

1. **全局单队列**: 所有CPU共享一个run_queue, 存在锁竞争瓶颈
2. **无CPU亲和性**: 进程可能在不同CPU间频繁迁移, 缓存失效
3. **负载不均衡**: 没有负载均衡机制, 可能导致某些CPU空闲而其他CPU繁忙

改进方案:

方案一：每CPU队列（Per-CPU Queue）

代码块

```
1  struct run_queue per_cpu_rq[NCPU]; // 每个CPU一个队列
2
3  static struct run_queue *get_cpu_rq(void) {
4      return &per_cpu_rq[cupid()];
5  }
```

- 优点：无锁竞争，缓存友好
- 缺点：需要实现负载均衡

方案二：分层调度（Hierarchical Scheduling）

- 全局队列 + 本地队列
- 周期性负载均衡（work stealing）
- CPU亲和性控制

所需改动：

1. 调度类扩展：

代码块

```
1  struct sched_class {
2      // ... 现有接口 ...
3      void (*load_balance)(struct run_queue *rq); // 负载均衡
4      int (*get_proc)(struct run_queue *rq, struct proc_struct
5      *procs_moved[]);
6  };
```

2. 进程结构修改：

代码块

```
1  struct proc_struct {
2      // ...
3      int cpu_affinity; // CPU亲和性掩码
4      int last_cpu; // 上次运行的CPU
5      uint64_t cache_hot_time; // 缓存热度时间戳
6  };
```

3. 调度核心修改：

- `schedule()`修改为使用本地队列
- 定时触发负载均衡（在时钟中断中）
- 支持进程迁移的同步机制

三、扩展练习Challenge 1：实现Stride Scheduling调度算法

1. 实现Stride Scheduling调度算法

`default_sched_stride.c`实现了 Stride Scheduling（步长调度）并把调度器封装为 `stride_sched_class`。采用斜堆（skew heap）作为优先队列（由 `USE_SKEW_HEAP` 宏控制），按 `lab6_stride` 的最小值选择下一个运行进程。每次选择后更新被选进程的 `lab6_stride += BIG_STRIDE / lab6_priority`，保证低优先级（较大优先级数值）得到更慢增长，从而分配更多 CPU 时间。

重要常量与比较函数：

- `#define BIG_STRIDE 0x7FFFFFFF`：一个很大的常数，作为步长基数，后续每次选中进程会增加该量除以其优先级，模拟“份额/比例”分配。
- `proc_stride_comp_f(void *a, void *b)`：
 - 将传入的 skheap 节点转换为 `proc_struct`（使用 `le2proc`）。
 - 比较 `p->lab6_stride` 与 `q->lab6_stride`，返回 -1/0/1 以供斜堆维护最小根性质。
 - 作用：斜堆中根节点保持 `lab6_stride` 最小的进程。

初始化：

- `stride_init(struct run_queue *rq)`：
 - `list_init(&(rq->run_list));`：初始化就绪链表（备用，当不使用斜堆时）。
 - `rq->lab6_run_pool = NULL;`：斜堆指针清空。
 - `rq->proc_num = 0;`：运行队列计数清零。
 - 说明：只做基本结构初始化，不设置 `max_time_slice`（由调用方负责）。

入队 (enqueue)：

- `stride_enqueue(struct run_queue *rq, struct proc_struct *proc)` 的主要步骤：
 - 将 `proc` 插入运行池：
 - 若 `USE_SKEW_HEAP`，调用 `skew_heap_insert(rq->lab6_run_pool, &(proc->lab6_run_pool), proc_stride_comp_f)` 并更新 `rq->lab6_run_pool`。
 - 否则使用链表 `list_add_before` 插入到 `rq->run_list`（代码里有备用分支）。
 - 设置 `proc->time_slice`：

- 如果 `proc->time_slice == 0` 或者大于 `rq->max_time_slice`，将其置为 `rq->max_time_slice`，确保新入队进程有合适时间片。

c. 设置 `proc->rq = rq`：记录所属运行队列。

d. `rq->proc_num++`：队列计数加一。

• 说明：

- 入队不改变 `lab6_stride` 或 `lab6_priority`；这些通常由进程创建时设置。
- 使用斜堆能以 $\log$ 时间插入并保持按 `lab6_stride` 排序。

出队 (dequeue)：

• `stride_dequeue(struct run_queue *rq, struct proc_struct *proc)`：

a. 若使用斜堆：`rq->lab6_run_pool = skew_heap_remove(rq->lab6_run_pool, &(proc->lab6_run_pool), proc_stride_comp_f);`

b. 否则从链表 `list_del_init` 删除节点。

c. `rq->proc_num--`：队列计数减一。

• 说明：移除时不调整 `proc->lab6_stride`；如果重新入队其 stride 保持原值（累积份额）。

选取下一个进程 (pick_next)：

• `stride_pick_next(struct run_queue *rq)`：

◦ 若斜堆为空返回 NULL。

◦ 否则取斜堆根：根节点为 `rq->lab6_run_pool`，用 `le2proc(rq->lab6_run_pool, lab6_run_pool)` 得到 `struct proc_struct *p`（代码在非斜堆分支中也有遍历实现来找 stride 最小者）。

◦ 防御性处理：若 `p->lab6_priority == 0`，将其设为 1（避免除零）。

◦ 更新步长：`p->lab6_stride += BIG_STRIDE / p->lab6_priority;`

◦ 返回 `p`。

• 设计意图：

- 在 stride 调度中，`stride` 值越小优先被选中；每被选中一次，增加步长，使其下一次被选的时间推后。增加量按 $1/\text{priority}$ 比例（priority 越大，增加越小，获得更多 CPU）。
- 使用大常数 `BIG_STRIDE` 使分数运算离散化并减小冲突概率。

处理时钟滴答 (proc_tick)：

• `stride_proc_tick(struct run_queue *rq, struct proc_struct *proc)`：

◦ 每个 tick 将 `proc->time_slice--`（如果 >0 ）。

◦ 当 `proc->time_slice == 0` 时，将 `proc->need_resched = 1`，触发上下文切换请求。

- 说明：

- 时间片由 `rq->max_time_slice` 在入队时分配；stride 调度通过 `lab6_stride` 控制“谁先被选”，而时间片控制单次执行长度。

调度类导出：

- `struct sched_class stride_sched_class`：
 - 将上面的函数按字段绑定：`.name`，`.init`，`.enqueue`，`.dequeue`，`.pick_next`，`.proc_tick`。
 - 这样内核中的调度子系统可以以统一接口使用该调度策略。

设计要点与边界/实现细节：

- 使用斜堆（skew heap）：
 - 优点：插入/删除/查找最小都较快（按斜堆性质），适合动态队列。
 - 代码通过 `USE_SKEW_HEAP` 宏允许用链表回退（便于调试或作业）。
- `lab6_stride` 的累积是单向的（不断增大），不会重置；长期运行会使 `lab6_stride` 值变大，使用 32-bit 时需注意溢出风险，但使用 `BIG_STRIDE`（0x7FFFFFFF）已接近 32 位上限，可能需要更精细的溢出处理（当前实现未处理）。
- `lab6_priority` 的含义：数值越大代表更高权重（更高优先级），因为增加量是 `BIG_STRIDE / priority`，priority 大 => 增加小 => 更频繁被选中。实现里防止 priority 为 0 导致除以 0。
- `time_slice` 与 `lab6_stride` 分离：
 - `time_slice` 控制单次运行长度（局部时间片），`lab6_stride` 控制长期的 CPU 份额分配（全局比例）。
- `enqueue` 未在插入时根据 `lab6_stride` 做任何初始化：
 - 假如新进程需要公平起始位置，创建时应给定合适的初始 `lab6_stride`（这在本文件外部设置）。
- `dequeue` 从结构中移除时不恢复或调整 `lab6_stride`，因此短暂停用再入队会保留原有份额位置。

完成代码后，执行 `make qemu`，输出如下：

```

++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
child pid 7, acc 940000, time 2010
child pid 6, acc 776000, time 2010
child pid 5, acc 624000, time 2010
child pid 4, acc 464000, time 2010
child pid 3, acc 312000, time 2020
main: pid 3, acc 312000, time 2020
main: pid 4, acc 464000, time 2020
main: pid 5, acc 624000, time 2020
main: pid 6, acc 776000, time 2020
main: pid 0, acc 940000, time 2020
main: wait pids over
sched result: 1 1 2 2 3
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:561:
initproc exit.

```

2. 简要说明如何设计实现“多级反馈队列调度算法”

总体设计思路

多级反馈队列调度器通过**维护多个不同优先级的就绪队列**，并根据进程在运行过程中的行为（是否用满时间片、是否主动阻塞）**动态调整其所在队列层级**，从而兼顾交互响应和系统吞吐量。

核心数据结构设计

1. 多级就绪队列

- 维护 `N` 个就绪队列 `Q[0...N-1]`
- `Q[0]` 优先级最高，`Q[N-1]` 最低
- 每个队列内部采用 **FIFO / RR**

2. 进程控制块（PCB）扩展字段

- `level`：当前所在队列层级
- `ticks_left`：当前时间片剩余 tick 数

3. 时间片数组

- `quantum[level]`：不同队列对应的的时间片长度（通常层级越低，时间片越大）

调度决策策略（Scheduling Policy）

1. 跨队列选择规则

- 总是从**最高非空队列**中选择进程运行

2. 队列内调度规则

- 同一队列内部采用 **Round-Robin**

反馈与优先级调整机制

1. 用满时间片（CPU-bound 行为）

- 若进程在其时间片内未阻塞且 `ticks_left == 0`
- 视为 CPU-bound
- 将进程 **降级** 到 `level + 1`（不超过最低层）

2. 未用满时间片（I/O-bound / 交互行为）

- 若进程在时间片结束前主动阻塞或让出 CPU
- 保持当前层级（或按设计提升一层）

3. 进程唤醒

- I/O 完成后，将进程放回其当前（或调整后的）队列尾部

时钟中断与抢占实现

1. 每次时钟中断：

- 对当前运行进程执行 `ticks_left--`

2. 若 `ticks_left == 0`：

- 触发抢占
- 根据反馈规则调整层级
- 重新加入对应队列

防止饥饿：优先级提升（Priority Boost）

- 每经过固定时间间隔 `BOOST_PERIOD`：
 - 将所有就绪进程提升到最高队列 `Q[0]`
 - 重置其 `level` 和 `ticks_left`
- 保证低优先级进程最终能获得 CPU

简化版伪代码

代码块

```
1  schedule():
2      for level = 0 .. N-1:
3          if Q[level] 非空:
4              选择 Q[level] 队头进程运行
```

代码块

```
1 on_timer_interrupt():
2     current.ticks_left--
3     if current.ticks_left == 0:
4         根据反馈规则调整 current.level
5         将 current 入队
6         触发调度
```

3. 简要说明为什么Stride算法中，经过足够多的时间片之后，每个进程分配到的时间片数目和优先级成正比

调度器始终选择pass最小的进程运行。经过足够多的时间片后，所有可运行进程的 `pass` 值会保持在同一数量级、相互接近（否则pass小的会被优先调度，直到追上来）。

设进程 i 运行了 N_i 个时间片： $\text{pass}_i \approx N_i \cdot \text{stride}_i$ ；

在稳定状态下： $N_i \cdot \text{stride}_i \approx N_j \cdot \text{stride}_j$ ；

代入 stride 定义： $N_i \cdot \frac{C}{\text{priority}_i} \approx N_j \cdot \frac{C}{\text{priority}_j}$ ；

消去常数 C ： $\frac{N_i}{N_j} \approx \frac{\text{priority}_i}{\text{priority}_j}$ 。

由于进程每运行一次其 pass 按与优先级成反比的 stride 增长，而调度器始终选择 pass 最小者运行，长期来看各进程的 pass 值趋于相等，从而其获得的时间片数与优先级成正比。

四、实验中的重要知识点

1. RR 调度算法

RR（Round-Robin）调度是一种基于时间片的抢占式调度算法，**依赖时钟中断**。所有就绪进程按循环队列顺序轮流使用 CPU，每次最多运行一个时间片。

数据结构

- 就绪队列：普通 FIFO 队列。

调度流程

- 从就绪队列队头取出进程。
- 运行至多一个时间片。

- 若进程未结束：放回队尾；若进程结束：直接移除。
- 调度下一个进程。

时间片大小的影响

- 时间片太大：行为接近 FCFS，响应慢；
- 时间片太小：上下文切换频繁，CPU 开销大。

2. Stride Scheduling 调度算法

Stride Scheduling（步长调度）是一种基于比例公平（proportional-share）的操作系统调度算法，目标是让各进程按照事先分配的“份额（share）”公平地占用 CPU，并且具有确定性、可预测的调度行为。

核心思想是每个进程按固定“步长”向前走，谁“走得最慢”，谁先运行。

三个核心概念

1. Share（份额）

- 表示进程希望获得的 CPU 比例
- share 越大，越“重要”

例如：

- A: share = 100
- B: share = 50
→ A 的 CPU 占用 $\approx$ B 的 2 倍

2. Stride（步长）

步长与份额**成反比**：

$$\text{stride} = \frac{\text{BIG_NUMBER}}{\text{share}}$$

- BIG_NUMBER 是一个很大的常数（如 10^6 ）
- share 大 $\rightarrow$ stride 小 $\rightarrow$ 走得慢 $\rightarrow$ 更容易被调度

3. Pass（通行值 / 累计值）

- 每个进程都有一个 `pass`
- 每运行一次，就加上自己的 stride
- 调度器每次选择 `pass` 最小的进程运行

调度规则（完整流程）

1. 初始化：每个进程 `pass = 0` ；
2. 每次调度：选择 **pass 最小** 的进程；
3. 该进程运行一个时间片；
4. 运行结束： `pass += stride` ；
5. 重复上述过程。

3. Linux CFS调度算法

完全公平调度（Completely Fair Scheduler, CFS）是Linux内核中用于进程调度的核心算法，其核心目标是实现**所有进程公平分享CPU资源**。以下是其核心机制和技术细节：

CFS是Linux内核（2.6.23+）默认的进程调度算法，旨在实现以下目标：

- **公平性**：所有可运行进程公平共享CPU时间；
- **低延迟**：快速响应交互式任务；
- **高效性**：避免传统时间片轮转的 $O(n)$ 复杂度。

公平性实现

1. 公平性设计

CFS通过**虚拟运行时间（vruntime）**实现公平性，每个进程的vruntime计算方式为：

```
vruntime = 实际运行时间 × 基准权重 / 进程权重
```

- **进程权重**：由进程的nice值决定，nice值越小（优先级越高），权重越大（`prio_to_weight` 数组转换）；
- **实际运行时间**：进程实际占用CPU的时间，由调度周期和权重比例分配。

2. 红黑树调度队列

CFS使用红黑树（自平衡二叉搜索树）存储可运行进程，**以vruntime为键值**，每次选择**最左侧节点**（vruntime最小的进程）运行，时间复杂度为 $O(\log n)$ 。

调度规则（完整流程）

1. 选择进程

- 从红黑树中取 **vruntime 最小** 的节点

2. 运行进程

- 运行一小段时间（不是固定时间片）

3. 更新 vruntime

```
delta_exec = 实际运行时间
```

```
p->vruntime += delta_exec × weight_factor
```

4. 重新插回红黑树

- 根据新的 vruntime 位置插入

五、其他重要知识点

其他常见的进程调度算法

1. FCFS (First-Come, First-Served, 先来先服务)

核心思想：谁先到就绪队列，谁先运行，直到结束或阻塞。

特点：**非抢占**，用普通 FIFO 队列。

2. SJF / SJN (Shortest Job First / Next, 最短作业优先)

核心思想：优先调度“预计运行时间最短”的进程。

两种形式：非抢占：SJF；抢占式：SRTF (Shortest Remaining Time First)。

优缺点：理论上平均等待时间最小，但是需要“预知未来”，长进程可能饥饿。

3. Priority Scheduling (优先级调度)

核心思想：优先级高的进程先运行。

特点：可抢占 / 非抢占，优先级可以是静态或动态。

经典问题：饥饿，即低优先级进程可能永远得不到 CPU。

经典解决方案：Aging（老化），即等待越久，优先级越高。